



# Movie Watchlist

Django + PostgreSQL Web Application

## Assignment 1

AST02.04 Full Stack Application Development · FSAD 2026

Student: st126005@ait.asia

Stack: Python 3.13 · Django 6.0 · PostgreSQL 18 · Bootstrap 5

## 1. Requirements Checklist

Every functional and technical requirement from the assignment brief, and where it is implemented. All items were verified in the running application.

| Requirement                          | Implemented by                    | Done |
|--------------------------------------|-----------------------------------|------|
| Homepage listing all movies          | movie_list view + movie_list.html | ✓    |
| Movies ordered by newest first       | Meta.ordering = ['-date_added']   | ✓    |
| Add a new movie                      | movie_create view                 | ✓    |
| Edit an existing movie               | movie_update view                 | ✓    |
| Delete a movie (with confirmation)   | movie_delete view                 | ✓    |
| Mark a movie as Watched / Unwatched  | movie_toggle_watched view         | ✓    |
| Search by movie title                | ?q= filter (title__icontains)     | ✓    |
| Title required; Rating 1–5 validated | model validators + ModelForm      | ✓    |
| Django + PostgreSQL, using the ORM   | settings.DATABASES + QuerySets    | ✓    |
| CRUD functionality                   | create / read / update / delete   | ✓    |
| HTML templates (Bootstrap)           | templates/ + Bootstrap 5          | ✓    |
| Movie registered in Django Admin     | movies/admin.py                   | ✓    |
| Automated tests                      | 9/9 passing                       | ✓    |
| Git version control                  | repository + commit history       | ✓    |

### GitHub Repository

<https://github.com/nyihtutoo/Movie-Watchlist>

## 2. Application Screenshots

Movie Watchlist

+ Add Movie

My Movies

4 total

Search by title...

Search

| Title      | Genre    | Year | Rating | Status    | Date Added         | Actions                                                |
|------------|----------|------|--------|-----------|--------------------|--------------------------------------------------------|
| Parasite   | Thriller | 2019 | ★★★★★  | Unwatched | 20 Aug 2026, 04:31 | <div>✓ Watch</div> <div>Edit</div> <div>Delete</div>   |
| The Batman | Action   | 2022 | ★★★★☆  | Unwatched | 20 Aug 2026, 04:31 | <div>✓ Watch</div> <div>Edit</div> <div>Delete</div>   |
| Dune       | Sci-Fi   | 2021 | ★★★★☆  | Watched   | 20 Aug 2026, 04:31 | <div>🔍 Unwatch</div> <div>Edit</div> <div>Delete</div> |
| Inception  | Sci-Fi   | 2010 | ★★★★★  | Watched   | 20 Aug 2026, 04:31 | <div>🔍 Unwatch</div> <div>Edit</div> <div>Delete</div> |

Full Stack Application Development · Assignment 1 · Django + PostgreSQL

Homepage — lists all movies ordered newest first, with genre, release year, star rating, Watched/Unwatched status badges, and per-row actions.

Movie Watchlist

+ Add Movie

My Movies

1 total

SearchClear

| Title | Genre  | Year | Rating | Status  | Date Added         | Actions                      |
|-------|--------|------|--------|---------|--------------------|------------------------------|
| Dune  | Sci-Fi | 2021 | ★★★★☆  | Watched | 20 Aug 2026, 04:31 | <div>UnwatchEditDelete</div> |

Full Stack Application Development · Assignment 1 · Django + PostgreSQL

Search by title — filtering the list with the query “dune” returns only the matching movie (case-insensitive).

Movie Watchlist

+ Add Movie

Add Movie

Title \*

Genre

Release year

Year the movie was released.

Personal rating

Your personal rating from 1 to 5.

☐ Watched

SaveCancel

Full Stack Application Development · Assignment 1 · Django + PostgreSQL

Add Movie — the create form (ModelForm) for a new entry.

 **Movie Watchlist**

[+ Add Movie](#)

### Add Movie

Title \*

This field is required.

Genre

Release year

Year the movie was released.

Personal rating

Your personal rating from 1 to 5.

Ensure this value is less than or equal to 5.

☐ Watched

[Save](#) [Cancel](#)

Full Stack Application Development · Assignment 1 · Django + PostgreSQL

Server-side validation — submitting an empty title and a rating of 9 is rejected: "This field is required" and "Ensure this value is less than or equal to 5".

 **Movie Watchlist**

[+ Add Movie](#)

### Edit Movie

Title \*

Genre

Release year

Year the movie was released.

Personal rating

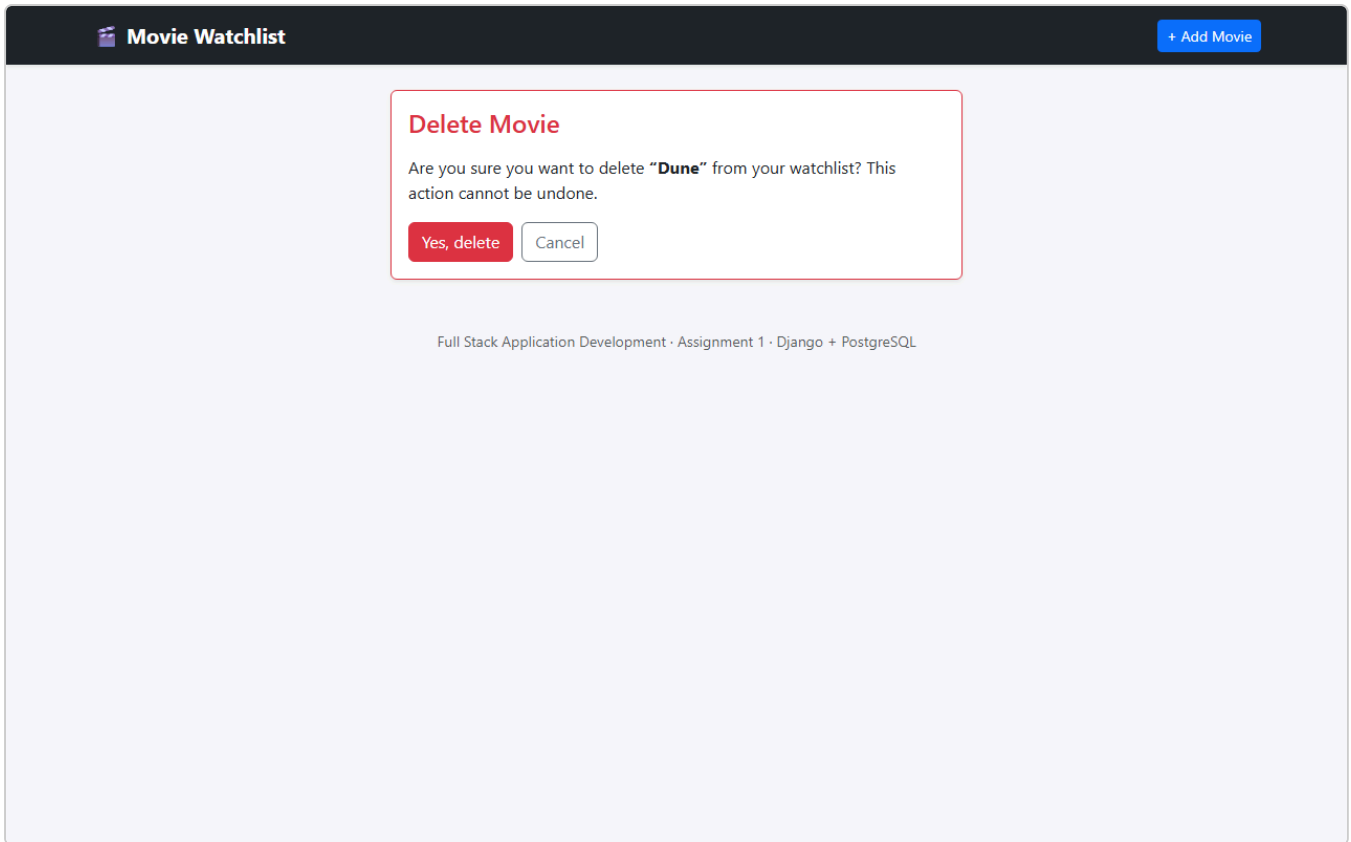
Your personal rating from 1 to 5.

☒ Watched

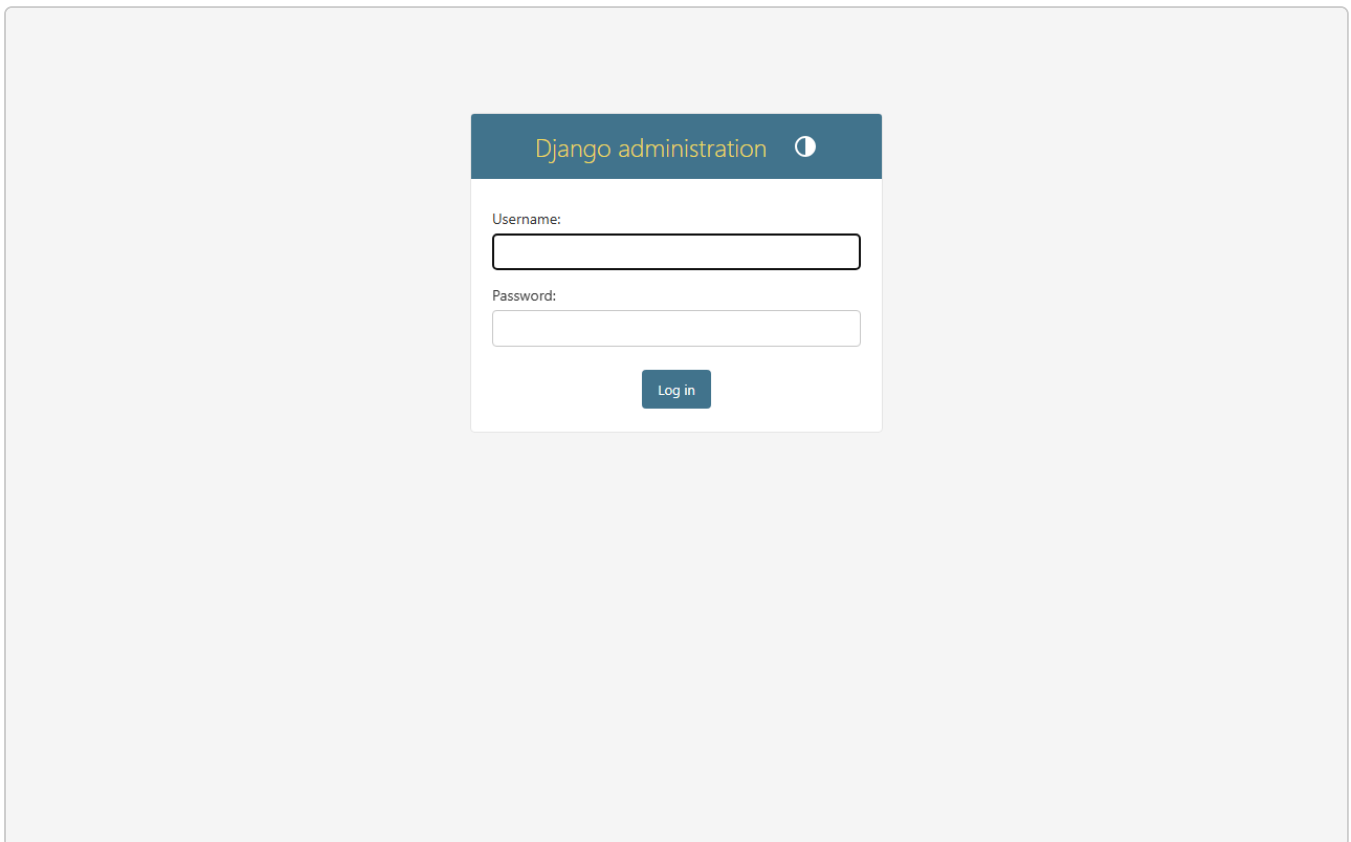
[Save](#) [Cancel](#)

Full Stack Application Development · Assignment 1 · Django + PostgreSQL

Edit Movie — the update form pre-populated with the existing movie's data.



*Delete confirmation — deletion requires an explicit confirmation (POST), preventing accidental removal.*



*Django Admin — the built-in admin login page.*

Django administration

WELCOME, ADMIN. [VIEW SITE](#) / [CHANGE PASSWORD](#) / [LOG OUT](#)

[Home](#) > [Movies](#) > Movies

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

Groups + Add

Users + Add

MOVIES

Movies + Add

Select movie to change

ADD MOVIE +

Q

Search

Action: 

-----

 Run 0 of 4 selected

| <input type="checkbox"/> | TITLE                      | GENRE    | RELEASE YEAR | PERSONAL RATING | WATCHED                             | DATE ADDED               |
|--------------------------|----------------------------|----------|--------------|-----------------|-------------------------------------|--------------------------|
| <input type="checkbox"/> | <a href="#">Parasite</a>   | Thriller | 2019         | 5               | <input type="checkbox"/>            | Aug. 20, 2026, 4:31 a.m. |
| <input type="checkbox"/> | <a href="#">The Batman</a> | Action   | 2022         | 3               | <input type="checkbox"/>            | Aug. 20, 2026, 4:31 a.m. |
| <input type="checkbox"/> | <a href="#">Dune</a>       | Sci-Fi   | 2021         | 4               | <input checked="" type="checkbox"/> | Aug. 20, 2026, 4:31 a.m. |
| <input type="checkbox"/> | <a href="#">Inception</a>  | Sci-Fi   | 2010         | 5               | <input checked="" type="checkbox"/> | Aug. 20, 2026, 4:31 a.m. |

4 movies

Save

FILTER

Show counts

↓ By watched

All

Yes

No

↓ By genre

All

Action

Sci-Fi

Thriller

Django Admin — the Movie model registered with a custom list display, “By watched” and “By genre” filters, search, and inline editable Watched field.

Django administration

WELCOME, ADMIN. [VIEW SITE](#) / [CHANGE PASSWORD](#) / [LOG OUT](#)

[Home](#) > [Movies](#) > [Movies](#) > Inception

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

Groups + Add

Users + Add

MOVIES

Movies + Add

Change movie

HISTORY

Title:

Inception

Genre:

Sci-Fi

Release year:

2010

Year the movie was released.

Personal rating:

5

Your personal rating from 1 to 5.

☒ Watched

SAVE

Save and add another

Save and continue editing

Delete

Django Admin — the Movie change form.

### 3. Source Code

---

#### movies/models.py

The Movie model — all six required fields with validation and newest-first ordering.

```
1  from django.core.validators import MaxValueValidator, MinValueValidator
2  from django.db import models
3
4
5  class Movie(models.Model):
6      """A single movie entry in the personal watchlist."""
7
8      title = models.CharField(max_length=200)
9      genre = models.CharField(max_length=100, blank=True)
10     release_year = models.PositiveIntegerField(
11         null=True,
12         blank=True,
13         validators=[MinValueValidator(1888), MaxValueValidator(2100)],
14         help_text="Year the movie was released.",
15     )
16     personal_rating = models.PositiveSmallIntegerField(
17         null=True,
18         blank=True,
19         validators=[MinValueValidator(1), MaxValueValidator(5)],
20         help_text="Your personal rating from 1 to 5.",
21     )
22     watched = models.BooleanField(default=False)
23     date_added = models.DateTimeField(auto_now_add=True)
24
25     class Meta:
26         # Homepage requirement: newest first.
27         ordering = ["-date_added"]
28
29     def __str__(self):
30         return self.title
```

## movies/forms.py

*MovieForm — a ModelForm shared by the add and edit views.*

```
1  from django import forms
2
3  from .models import Movie
4
5
6  class MovieForm(forms.ModelForm):
7      """Form used for both adding and editing a movie."""
8
9      class Meta:
10         model = Movie
11         fields = ["title", "genre", "release_year", "personal_rating", "watched"]
12         widgets = {
13             "title": forms.TextInput(
14                 attrs={"class": "form-control", "placeholder": "e.g. Inception"}
15             ),
16             "genre": forms.TextInput(
17                 attrs={"class": "form-control", "placeholder": "e.g. Sci-Fi"}
18             ),
19             "release_year": forms.NumberInput(
20                 attrs={"class": "form-control", "placeholder": "e.g. 2010"}
21             ),
22             "personal_rating": forms.NumberInput(
23                 attrs={"class": "form-control", "min": 1, "max": 5}
24             ),
25             "watched": forms.CheckboxInput(attrs={"class": "form-check-input"}),
26         }
27
28     def clean_title(self):
29         title = self.cleaned_data.get("title", "").strip()
30         if not title:
31             raise forms.ValidationError("Title is required.")
32         return title
```

## movies/views.py

Views — CRUD, search, and the watched-toggle, using the Django ORM.

```
1  from django.contrib import messages
2  from django.db.models import Q
3  from django.shortcuts import get_object_or_404, redirect, render
4
5  from .forms import MovieForm
6  from .models import Movie
7
8
9  def movie_list(request):
10     """Homepage: List all movies (newest first) with title search."""
11     query = request.GET.get("q", "").strip()
12     movies = Movie.objects.all() # Meta.ordering already sorts newest first
13     if query:
14         movies = movies.filter(Q(title__icontains=query))
15     context = {"movies": movies, "query": query}
16     return render(request, "movies/movie_list.html", context)
17
18
19  def movie_create(request):
20     """Add a new movie."""
21     if request.method == "POST":
22         form = MovieForm(request.POST)
23         if form.is_valid():
24             movie = form.save()
25             messages.success(request, f'{movie.title} was added to your watchlist.')
26             return redirect("movies:movie_list")
27     else:
28         form = MovieForm()
29     context = {"form": form, "title": "Add Movie"}
30     return render(request, "movies/movie_form.html", context)
31
32
33  def movie_update(request, pk):
34     """Edit an existing movie."""
35     movie = get_object_or_404(Movie, pk=pk)
36     if request.method == "POST":
37         form = MovieForm(request.POST, instance=movie)
38         if form.is_valid():
39             form.save()
40             messages.success(request, f'{movie.title} was updated.')
41             return redirect("movies:movie_list")
42     else:
43         form = MovieForm(instance=movie)
44     context = {"form": form, "title": "Edit Movie", "movie": movie}
45     return render(request, "movies/movie_form.html", context)
46
47
48  def movie_delete(request, pk):
49     """Delete a movie (confirmation on GET, delete on POST)."""
50     movie = get_object_or_404(Movie, pk=pk)
51     if request.method == "POST":
52         title = movie.title
53         movie.delete()
54         messages.success(request, f'{title} was removed from your watchlist.')
55         return redirect("movies:movie_list")
56     context = {"movie": movie}
57     return render(request, "movies/movie_confirm_delete.html", context)
58
59
60  def movie_toggle_watched(request, pk):
61     """Mark a movie as Watched or Unwatched."""
62     movie = get_object_or_404(Movie, pk=pk)
63     if request.method == "POST":
64         movie.watched = not movie.watched
65         movie.save(update_fields=["watched"])
66         state = "watched" if movie.watched else "unwatched"
67         messages.success(request, f'{movie.title} marked as {state}.')
68     return redirect("movies:movie_list")
```

## movies/urls.py

App-level URL routing.

```
1  from django.urls import path
2
3  from . import views
4
5  app_name = "movies"
6
7  urlpatterns = [
8      path("", views.movie_list, name="movie_list"),
9      path("add/", views.movie_create, name="movie_create"),
10     path("<int:pk>/edit/", views.movie_update, name="movie_update"),
11     path("<int:pk>/delete/", views.movie_delete, name="movie_delete"),
12     path("<int:pk>/toggle/", views.movie_toggle_watched, name="movie_toggle_watched"),
13 ]
```

## config/urls.py

Project-level URL routing.

```
1  from django.contrib import admin
2  from django.urls import include, path
3
4  urlpatterns = [
5      path("admin/", admin.site.urls),
6      path("", include("movies.urls")),
7  ]
```

## movies/admin.py

Registering the Movie model in the Django Admin.

```
1  from django.contrib import admin
2
3  from .models import Movie
4
5
6  @admin.register(Movie)
7  class MovieAdmin(admin.ModelAdmin):
8      list_display = (
9          "title",
10         "genre",
11         "release_year",
12         "personal_rating",
13         "watched",
14         "date_added",
15     )
16     list_filter = ("watched", "genre")
17     search_fields = ("title",)
18     list_editable = ("watched",)
19     ordering = ("-date_added",)
```

## config/settings.py

Project settings — installed apps, templates, and the PostgreSQL database configuration (secrets read from a .env file).

```
1  """
2  Django settings for config project.
3
4  Generated by 'django-admin startproject' using Django 6.0.8.
5
6  For more information on this file, see
7  https://docs.djangoproject.com/en/6.0/topics/settings/
8
9  For the full list of settings and their values, see
10 https://docs.djangoproject.com/en/6.0/ref/settings/
11 """
12
13 import os
14 from pathlib import Path
15
16 from dotenv import load_dotenv
17
18 # Build paths inside the project like this: BASE_DIR / 'subdir'.
19 BASE_DIR = Path(__file__).resolve().parent.parent
20
21 # Load environment variables from a .env file (never commit .env to git).
22 load_dotenv(BASE_DIR / ".env")
23
24
25 # Quick-start development settings - unsuitable for production
26 # See https://docs.djangoproject.com/en/6.0/howto/deployment/checklist/
27
28 # SECURITY WARNING: keep the secret key used in production secret!
29 SECRET_KEY = os.getenv(
30     "DJANGO_SECRET_KEY",
31     "django-insecure-8h*0#&5&9eyo75c$me^18xv^+ns56gnsv)*6_ib=!jx@x&b$_s",
32 )
33
34 # SECURITY WARNING: don't run with debug turned on in production!
35 DEBUG = os.getenv("DJANGO_DEBUG", "True") == "True"
36
37 ALLOWED_HOSTS = os.getenv("DJANGO_ALLOWED_HOSTS", "localhost,127.0.0.1").split(",")
38
39
40 # Application definition
41
42 INSTALLED_APPS = [
43     'django.contrib.admin',
44     'django.contrib.auth',
45     'django.contrib.contenttypes',
46     'django.contrib.sessions',
47     'django.contrib.messages',
48     'django.contrib.staticfiles',
49     'movies',
50 ]
51
52 MIDDLEWARE = [
53     'django.middleware.security.SecurityMiddleware',
54     'django.contrib.sessions.middleware.SessionMiddleware',
55     'django.middleware.common.CommonMiddleware',
56     'django.middleware.csrf.CsrfViewMiddleware',
57     'django.contrib.auth.middleware.AuthenticationMiddleware',
58     'django.contrib.messages.middleware.MessageMiddleware',
59     'django.middleware.clickjacking.XFrameOptionsMiddleware',
60 ]
61
62 ROOT_URLCONF = 'config.urls'
63
64 TEMPLATES = [
65     {
66         'BACKEND': 'django.template.backends.django.DjangoTemplates',
67         'DIRS': [BASE_DIR / 'templates'],
68         'APP_DIRS': True,
```

```

69         'OPTIONS': {
70             'context_processors': [
71                 'django.template.context_processors.request',
72                 'django.contrib.auth.context_processors.auth',
73                 'django.contrib.messages.context_processors.messages',
74             ],
75         },
76     },
77 ]
78
79 WSGI_APPLICATION = 'config.wsgi.application'
80
81
82 # Database
83 # https://docs.djangoproject.com/en/6.0/ref/settings/#databases
84
85 DATABASES = {
86     'default': {
87         'ENGINE': 'django.db.backends.postgresql',
88         'NAME': os.getenv('DB_NAME', 'movie_watchlist'),
89         'USER': os.getenv('DB_USER', 'movie_user'),
90         'PASSWORD': os.getenv('DB_PASSWORD', ''),
91         'HOST': os.getenv('DB_HOST', 'localhost'),
92         'PORT': os.getenv('DB_PORT', '5432'),
93     }
94 }
95
96
97 # Password validation
98 # https://docs.djangoproject.com/en/6.0/ref/settings/#auth-password-validators
99
100 AUTH_PASSWORD_VALIDATORS = [
101     {
102         'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
103     },
104     {
105         'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
106     },
107     {
108         'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
109     },
110     {
111         'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
112     },
113 ]
114
115
116 # Internationalization
117 # https://docs.djangoproject.com/en/6.0/topics/i18n/
118
119 LANGUAGE_CODE = 'en-us'
120
121 TIME_ZONE = 'Asia/Bangkok'
122
123 USE_I18N = True
124
125 USE_TZ = True
126
127
128 # Static files (CSS, JavaScript, Images)
129 # https://docs.djangoproject.com/en/6.0/howto/static-files/
130
131 STATIC_URL = 'static/'
132
133 # Default primary key field type
134 # https://docs.djangoproject.com/en/6.0/ref/settings/#default-auto-field
135 DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
136
137 # Map Django message levels to Bootstrap alert classes.
138 from django.contrib.messages import constants as messages # noqa: E402
139
140 MESSAGE_TAGS = {

```

```
141     messages.DEBUG: 'secondary',
142     messages.INFO: 'info',
143     messages.SUCCESS: 'success',
144     messages.WARNING: 'warning',
145     messages.ERROR: 'danger',
146 }
```

## movies/tests.py

Automated tests for the model and the views (9 tests).

```
1  from django.test import TestCase
2  from django.urls import reverse
3
4  from .models import Movie
5
6
7  class MovieModelTests(TestCase):
8      def test_str_returns_title(self):
9          movie = Movie.objects.create(title="Inception")
10         self.assertEqual(str(movie), "Inception")
11
12     def test_default_watched_is_false(self):
13         movie = Movie.objects.create(title="Dune")
14         self.assertFalse(movie.watched)
15
16     def test_ordering_is_newest_first(self):
17         first = Movie.objects.create(title="Old")
18         second = Movie.objects.create(title="New")
19         titles = list(Movie.objects.values_list("title", flat=True))
20         self.assertEqual(titles, ["New", "Old"])
21         self.assertEqual(Movie.objects.first(), second)
22         self.assertEqual(Movie.objects.last(), first)
23
24
25     class MovieViewTests(TestCase):
26         def test_list_view_status_ok(self):
27             response = self.client.get(reverse("movies:movie_list"))
28             self.assertEqual(response.status_code, 200)
29
30         def test_create_movie(self):
31             response = self.client.post(
32                 reverse("movies:movie_create"),
33                 {"title": "Interstellar", "genre": "Sci-Fi", "release_year": 2014,
34                  "personal_rating": 5, "watched": False},
35             )
36             self.assertEqual(response.status_code, 302)
37             self.assertTrue(Movie.objects.filter(title="Interstellar").exists())
38
39         def test_update_movie(self):
40             movie = Movie.objects.create(title="Old Title")
41             self.client.post(
42                 reverse("movies:movie_update", args=[movie.pk]),
43                 {"title": "New Title", "genre": "", "release_year": "",
44                  "personal_rating": "", "watched": False},
45             )
46             movie.refresh_from_db()
47             self.assertEqual(movie.title, "New Title")
48
49         def test_delete_movie(self):
50             movie = Movie.objects.create(title="To Delete")
51             self.client.post(reverse("movies:movie_delete", args=[movie.pk]))
52             self.assertFalse(Movie.objects.filter(pk=movie.pk).exists())
53
54         def test_toggle_watched(self):
55             movie = Movie.objects.create(title="Toggle Me", watched=False)
56             self.client.post(reverse("movies:movie_toggle_watched", args=[movie.pk]))
57             movie.refresh_from_db()
58             self.assertTrue(movie.watched)
59
60         def test_search_by_title(self):
61             Movie.objects.create(title="The Matrix")
62             Movie.objects.create(title="Titanic")
63             response = self.client.get(reverse("movies:movie_list"), {"q": "matrix"})
64             self.assertContains(response, "The Matrix")
65             self.assertNotContains(response, "Titanic")
```

## templates/movies/movie\_list.html

The homepage template (Django Template Language + Bootstrap).

```
{% extends "base.html" %}

{% block title %}My Movies · Movie Watchlist{% endblock %}

{% block content %}
<div class="d-flex justify-content-between align-items-center mb-3">
  <h1 class="h3 mb-0">My Movies</h1>
  <span class="badge bg-secondary">{{ movies|length }} total</span>
</div>

<form method="get" class="mb-4">
  <div class="input-group">
    <input type="text" name="q" value="{{ query }}"
      class="form-control" placeholder="Search by title...">
    <button class="btn btn-outline-primary" type="submit">Search</button>
    {% if query %}
      <a class="btn btn-outline-secondary" href="{% url 'movies:movie_list' %}">
        Clear
      </a>
    {% endif %}
  </div>
</form>

{% if movies %}
<div class="table-responsive">
  <table class="table table-hover align-middle bg-white shadow-sm rounded">
    <thead class="table-dark">
      <tr>
        <th>Title</th>
        <th>Genre</th>
        <th>Year</th>
        <th>Rating</th>
        <th>Status</th>
        <th>Date Added</th>
        <th class="text-end">Actions</th>
      </tr>
    </thead>
    <tbody>
      {% for movie in movies %}
      <tr>
        <td class="fw-semibold">{{ movie.title }}</td>
        <td>{{ movie.genre|default:"-"}</td>
        <td>{{ movie.release_year|default:"-"}</td>
        <td>
          {% if movie.personal_rating %}
            {% for _ in "12345"|make_list %}{% if forloop.counter <= movie.personal_rating %}★{% else %}☆{% endif %}{% endfor %}
          {% else %}
            -
          {% endif %}
        </td>
        <td>
          {% if movie.watched %}
            <span class="badge bg-success">Watched</span>
          {% else %}
            <span class="badge bg-warning text-dark">Unwatched</span>
          {% endif %}
        </td>
        <td class="text-muted small">{{ movie.date_added|date:"d M Y, H:i" }}</td>
        <td class="text-end">
          <form method="post"
            action="{% url 'movies:movie_toggle_watched' movie.pk %}"
            class="d-inline">
            {% csrf_token %}
            <button class="btn btn-sm btn-outline-success" type="submit"
              title="Toggle watched">
              {% if movie.watched %}↶ Unwatch{% else %}✓ Watch{% endif %}
            </button>
          </td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
</div>
{% endif %}
```

```

3         </form>
5         <a class="btn btn-sm btn-outline-primary"
3             href="{% url 'movies:movie_update' movie.pk %}">Edit</a>
6         <a class="btn btn-sm btn-outline-danger"
3             href="{% url 'movies:movie_delete' movie.pk %}">Delete</a>
7     </td>
3 </tr>
8 {% endfor %}
3 </tbody>
9 </table>
4 </div>
0 {% else %}
4 <div class="text-center text-muted py-5 bg-white rounded shadow-sm">
1     {% if query %}
4         <p class="mb-1">No movies match “{{ query }}”.</p>
2         <a href="{% url 'movies:movie_list' %}">Show all movies</a>
4     {% else %}
3         <p class="mb-1">Your watchlist is empty.</p>
4         <a href="{% url 'movies:movie_create' %}">Add your first movie</a>
4     {% endif %}
4 </div>
5 {% endif %}
4 {% endblock %}
6
4
7
4
8
4
9
5
0
5
1
5
2
5
3
5
4
5
5
5
6
5
7
5
8
5
9
6
0
6
1
6
2
6
3
6
4
6
5
6
6
7
6
8
6
9
7
0

```

|   |  |
|---|--|
| 7 |  |
| 1 |  |
| 7 |  |
| 2 |  |
| 7 |  |
| 3 |  |
| 7 |  |
| 4 |  |
| 7 |  |
| 5 |  |
| 7 |  |
| 6 |  |
| 7 |  |
| 7 |  |
| 7 |  |
| 8 |  |
| 7 |  |
| 9 |  |
| 8 |  |
| 0 |  |
| 8 |  |
| 1 |  |
| 8 |  |
| 2 |  |
| 8 |  |
| 3 |  |
| 8 |  |
| 4 |  |
| 8 |  |
| 5 |  |
| 8 |  |
| 6 |  |
| 8 |  |
| 7 |  |
| 8 |  |
| 8 |  |
| 8 |  |
| 9 |  |
| 9 |  |
| 0 |  |